LESSON PLAN · PYTHON (GR 6–8) INTERMEDIATE: PYTHON GAME CODERS · LEVEL 2

Lesson 1: Functions That Give Answers Back

Module 1 | 60 minutes | parameters and return | Grades 7–11 · ages 12–16

Objective: Write your own functions with several parameters and a return value, and use them to build a cricket score calculator.

Success criteria

- I can write a function with two or more parameters.
- I can return a value and store it in a variable.
- I can explain the difference between print and return.
- I can use a default value for a parameter.

Key vocabulary

- **parameter**: A name inside the def brackets that receives a value.
- **argument**: The actual value you pass when you call the function.
- **return**: Sends a value back to the line that called the function.
- **None**: Python's value for "nothing"; what a function gives back with no return.
- **default value**: A value a parameter uses if you leave it out, like overs=20.
- **strike rate**: Runs scored per 100 balls: runs × 100 ÷ balls.

You will need

- Projector with Kitabash (lesson video, Python Lab)
- Devices: 1 per learner, with a keyboard
- Printed worksheet (optional)
- Project files: cricket_helpers.py

Timed lesson plan (60 min)

Time	Stage	What happens
5 min	**Warm-up game: Function Machine**	Play Function Machine (unplugged). Learners call the machine with arguments and race to guess the rule. Winner becomes the next machine.
5 min	**Hook and video**	Show the hook. Chat poll: "What will double(21) print?" Play the video with captions; pause at "print shows, return gives back" and ask what a holds.
10 min	**Live demo and build-along**	Run the live demo, then everyone types the cricket helper functions. Thumbs check after each step: "Do you see 150.0?"
8 min	**Challenge, predict and match**	Total runs function challenge, the predict-the-output and the word match. Quick competition: first three correct predictions in the chat

		get a “Six!” shout-out.
15 min	**Project in breakout pairs**	Pair programming in breakout rooms: the driver types, the navigator checks every function has return; swap every 5 minutes. Build the cricket score calculator.
6 min	**Debug race**	Race to fix “the average that vanished”. The first learner to explain both bugs out loud wins.
7 min	**Show-and-tell**	Two or three learners share their screen and run the calculator with a real innings. The class checks one answer with mental maths and gives two stars and a wish.
4 min	**Wrap-up and exit ticket**	Fist-to-five poll on the success criteria. Exit ticket: the square(n) function.

Unplugged warm-up

Function Machine (5 min game): the teacher is a “function” with a secret rule (for example return n * 3 + 1). Learners call it out loud with an argument, like “machine of 4!”, and the teacher returns the answer on the whiteboard. The first learner to write the rule as def machine(n): return ... wins and becomes the next machine.

Teaching the idea

In Level 1 learners wrote functions that only printed. Today functions take several parameters and return an answer. Parameters are the names in the def brackets; arguments are the values passed in. return sends a value back to the caller, so it can be stored, printed or used in more maths. A function without return gives back None, which is the most common bug today. Default values like overs=20 make a parameter optional.

Analogy: A function is like a juice stall: you hand over the fruit (arguments), the stall does its work inside, and return hands you back a glass of juice. print would be the stall shouting “juice!” without giving you anything to drink.

Live demo: 1) Type def double(n): with return n * 2, then print(double(21)); run: 42. 2) Change return to print(n * 2) and run: point to 42 and then None, and ask where None came from. Put return back. 3) Type def strike_rate(runs, balls): return runs * 100 / balls and call strike_rate(45, 30): 150.0. 4) Swap the arguments to strike_rate(30, 45) and ask why the answer changed: order matters. 5) Add round(..., 2) and try strike_rate(10, 3). 6) Show def run_rate(runs, overs=20): and call it with and without the second argument.

Questions to ask

Ask	Listen for
What is the difference between a parameter and an argument?	The parameter is the name in def; the argument is the value you pass when calling.

What does a function give back if it has no return?	None.
Why is return better than print for strike_rate?	The answer can be stored and used again, for example in an if or more maths.
Does the order of arguments matter?	Yes. The first argument goes into the first parameter.
What does overs=20 mean in def run_rate(runs, overs=20)?	If you leave overs out, it uses 20.

Differentiation

Support
Give a template card: def name(a, b): / return a ? b. Let learners write strike_rate first with the formula given, and test it with 50 runs from 50 balls (answer 100).

Stretch
Write required_rate(target, runs, overs_left) that returns the runs per over a team needs to win, and use it to print a chase message.

Common misconceptions

Misconception	How to fix it
print inside a function gives the answer back	print only shows it on screen. Use return to give it back.
Code after return still runs	return ends the function immediately.
The argument must have the same name as the parameter	Any value or variable can be passed; it is copied into the parameter.
Defining a function runs it	The function only runs when you call it.

Assessment

Use the Python Lab results (auto-checked), the quick checks and the Probe answers. Written answers arrive in your review queue.

Exit ticket
Write a function square(n) that returns n times n, and one line that prints square(9).

Homework
Use your calculator on a real match scorecard from the newspaper or a cricket site with someone at home. Add a function economy(runs_given, overs) for bowlers.

Answer key — Kitabash activities

1. Python Lab: total runs function (perform · python lab)

Goals checked: Returns the right total · Prints 17 · Uses def and return

One solution:

```
def total_runs(fours, sixes, singles):
    runs = fours * 4 + sixes * 6 + singles
    return runs

print(total_runs(2, 1, 3))
```

2. Predict the output: bonus points (perform · code output)

3. Match each function word to its meaning (perform · matching)

def → Starts a new function · parameter → A name in the def brackets that receives a value · argument → The value you pass in when you call it · return → Sends an answer back to the caller · None → What you get back when there is no return

4. Project: cricket score calculator (perform · python lab)

Goals checked: strike_rate returns the right answer · run_rate returns the right answer · Prints both rates · Uses functions and input

Example program:

```
# Cricket score calculator
# Made by: Kabir

def strike_rate(runs, balls):
    return round(runs * 100 / balls, 2)

def run_rate(runs, overs):
    return round(runs / overs, 2)

def verdict(sr):
    if sr >= 150:
        return "Explosive batting!"
    elif sr >= 100:
        return "Good, quick scoring."
    else:
        return "Steady innings."

print("=== KITABASH CRICKET CALCULATOR ===")
runs = int(input("Batter's runs: "))
balls = int(input("Balls faced: "))
sr = strike_rate(runs, balls)
print(f"Strike rate: {sr}")
print(verdict(sr))

team_runs = int(input("Team total runs: "))
overs = int(input("Overs played: "))
print(f"Run rate: {run_rate(team_runs, overs)} runs per over")
```

5. Debug it: the average that vanished (probe · python lab)

Goals checked: Returns the average · Prints 45.0

One solution:

```
def average(a, b, c):
    total = a + b + c
    avg = total / 3
    return avg

print(average(30, 45, 60))
```

Answer key — worksheet

1. **Key words:** parameter = A name inside the def brackets that receives a value. · argument = The actual value you pass when you call the function. · return = Sends a value back to the line that

called the function. · None = Python’s value for “nothing”; what a function gives back with no return.

2. **Plan your program:** Goals: Returns the right total; Prints 17; Uses def and return. One solution: def total_runs(fours, sixes, singles): runs = fours * 4 + sixes * 6 + singles return runs print(total_runs(2, 1, 3))
3. **Predict the output: bonus points:** A. 30
4. **Match each function word to its meaning:** def → Starts a new function · parameter → A name in the def brackets that receives a value · argument → The value you pass in when you call it · return → Sends an answer back to the caller · None → What you get back when there is no return
5. **Plan your program:** Goals: Returns the average; Prints 45.0. One solution: def average(a, b, c): total = a + b + c avg = total / 3 return avg print(average(30, 45, 60))

Kitabash original curriculum.